Análise e Desenvolvimento de Sistemas
Unidade Curricular: Algoritmos e Programação avançada
Professor: Alex R. Marques

Gerenciador de dados:
Manipulação e persistência de Arquivos CSV e JSON

Tátila França de Sousa (26.1.20352)

Balsas MA

1. INTRODUÇÃO E OBJETIVOS

O presente projeto consiste no desenvolvimento de um sistema de gerenciamento de dados utilizando a linguagem de programação Python. A aplicação foi desenvolvida com a finalidade de realizar operações de leitura, escrita, conversão e armazenamento de informações em arquivos externos nos formatos CSV e JSON.

O projeto também contempla mecanismos de backup automático e tratamento de exceções, buscando garantir maior segurança e integridade das informações armazenadas.

Objetivo geral

Desenvolver um módulo de gerenciamento de persistência de dados capaz de manipular arquivos CSV e JSON, realizar conversões entre esses formatos e preservar as informações por meio de cópias de segurança.

Objetivos específicos

- Implementar a leitura de arquivos CSV e JSON.
- Desenvolver mecanismos de conversão entre os formatos.
- Utilizar o gerenciador de contexto "with open()".
- Implementar backup automático antes das alterações.
- Aplicar tratamento de exceções durante as operações.
- Testar as funcionalidades desenvolvidas.

2. DESENVOLVIMENTO

O desenvolvimento do projeto Gerenciador de Dados foi realizado utilizando a linguagem de programação Python, com o objetivo de criar um sistema capaz de realizar operações

com arquivos nos formatos CSV e JSON.

Para organizar o código e facilitar sua manutenção, o projeto foi dividido em diferentes módulos, cada um responsável por uma funcionalidade específica.

2.1 Estrutura do projeto

O sistema foi organizado nos seguintes arquivos:

- main.py: arquivo principal, responsável por executar o programa e realizar os testes das funcionalidades.
- leitura.py: responsável pela leitura de arquivos CSV e JSON.
- conversao.py: responsável pela conversão de arquivos CSV para JSON e de JSON para CSV.
- backup.py: responsável pela criação de cópias de segurança dos arquivos antes das operações de conversão.
- tratamento_erros.py: responsável pelo tratamento de exceções e identificação de possíveis erros durante a execução.

Também foram utilizadas pastas para organizar os arquivos de dados, testes e backups gerados pelo sistema.

2.2 Funcionamento do sistema

O funcionamento do programa foi desenvolvido por meio da integração dos módulos. Inicialmente, o sistema realiza a leitura dos arquivos, permitindo acessar informações armazenadas nos formatos CSV e JSON.

Em seguida, são realizadas as operações de conversão entre os formatos, possibilitando transformar os dados sem perder sua organização.

O projeto também conta com um mecanismo de backup automático, que cria cópias de segurança antes de determinadas operações, contribuindo para a preservação dos arquivos originais.

Além disso, foi implementado o tratamento de erros, permitindo identificar situações como arquivos inexistentes e evitar que problemas interrompam inesperadamente a execução do programa.

2.3 Organização e implementação

Durante a implementação, foram utilizadas funções para separar as responsabilidades de cada operação, facilitando a reutilização do código e a organização das funcionalidades.

A utilização de módulos permitiu estruturar o projeto de maneira mais organizada, tornando o código mais compreensível e facilitando futuras alterações e melhorias.

3. IMPLEMENTAÇÃO

A implementação do Gerenciador de Dados foi realizada utilizando a linguagem Python e seus módulos nativos, permitindo desenvolver funcionalidades de leitura, conversão, armazenamento e proteção de arquivos nos formatos CSV e JSON.

Para facilitar a organização e a manutenção do código, foram utilizadas funções, estruturas condicionais, tratamento de exceções e operações de manipulação de arquivos.

3.1 Bibliotecas utilizadas

Durante a implementação, foram utilizadas diferentes bibliotecas e módulos do Python:

- csv: utilizado para realizar a leitura e escrita de arquivos no formato CSV.
- json: utilizado para interpretar, armazenar e converter informações no formato JSON.
- shutil: utilizado para realizar cópias de segurança dos arquivos.
- os: utilizado para verificar a existência de arquivos e criar diretórios.
- datetime: utilizado para gerar identificadores de data e horário nos nomes dos backups.

Esses recursos permitiram desenvolver as funcionalidades necessárias sem a utilização de bibliotecas externas.

3.2 Leitura e manipulação de arquivos

A leitura dos arquivos foi implementada por meio das funções "ler_csv()" e "ler_json()", desenvolvidas no módulo "leitura.py".

Para acessar os arquivos, foi utilizado o comando "with open()", que permite abrir e fechar os arquivos automaticamente após a execução das operações.

Também foi utilizada a codificação UTF-8, permitindo trabalhar com caracteres especiais da língua portuguesa.

3.3 Conversão entre formatos

No módulo "conversao.py", foram desenvolvidas duas funções principais:

- "converter_csv_para_json()": realiza a leitura dos dados CSV e os armazena em um novo arquivo JSON.
- "converter_json_para_csv()": realiza a leitura dos dados JSON e os transforma em um arquivo CSV.

Para essas operações, foram utilizados recursos como "csv.DictReader()", "csv.DictWriter()", "json.load()" e "json.dump()".

Essas ferramentas possibilitam manipular os registros e preservar a organização das informações durante as conversões.

3.4 Implementação do backup automático

A funcionalidade de backup foi desenvolvida no módulo "backup.py", utilizando o módulo "shutil".

A função "criar_backup()" verifica a existência do arquivo de destino e, caso ele já exista, cria uma cópia de segurança antes de sua atualização.

Os backups são armazenados em um diretório específico, com nomes que incluem informações de data e horário, evitando a substituição de cópias anteriores.

3.5 Tratamento de exceções

O tratamento de exceções foi implementado no módulo "tratamento_erros.py", com o objetivo de identificar e apresentar mensagens relacionadas aos possíveis erros durante a execução.

Foram considerados problemas como arquivos inexistentes, falta de permissão de acesso, erros de codificação e arquivos JSON com estrutura inválida.

Para isso, foram utilizadas as estruturas "try" e "except", permitindo capturar exceções e apresentar mensagens mais compreensíveis.

3.6 Integração dos módulos

A integração das funcionalidades foi realizada por meio do arquivo "main.py", responsável por importar as funções dos demais módulos e executar os testes do sistema.

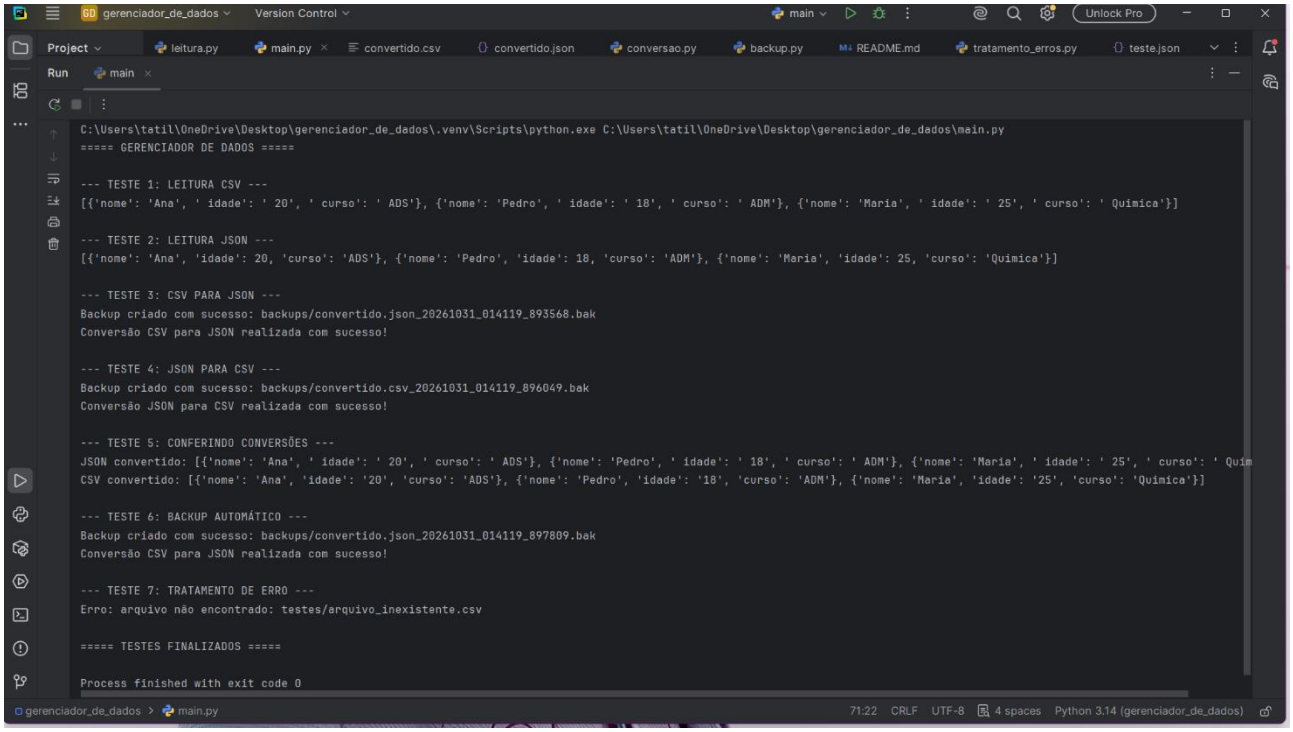
Dessa forma, foi possível reunir as operações de leitura, conversão, backup e tratamento de exceções em uma única execução, facilitando a verificação do funcionamento geral da aplicação.

4. TESTES E EVIDÊNCIAS

Os testes foram realizados com o objetivo de verificar o funcionamento das funcionalidades desenvolvidas no projeto Gerenciador de Dados. Foram analisadas a leitura de arquivos CSV e JSON, a conversão entre os formatos, a criação de backups e o tratamento de erros.

4.1 Teste de leitura de arquivo CSV

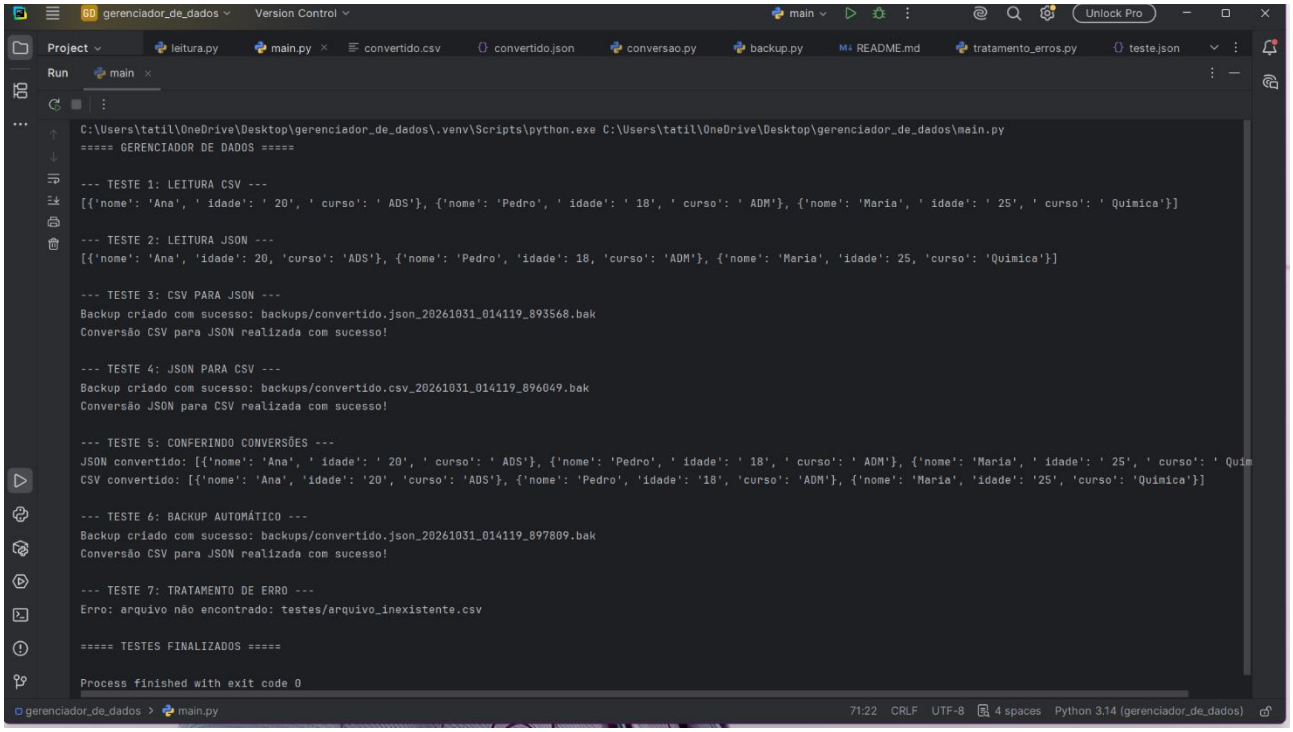
O primeiro teste teve como objetivo verificar se o sistema conseguia abrir e realizar a leitura de arquivos no formato CSV. A função "ler_csv()" foi executada para acessar os dados armazenados no arquivo e apresentá-los no terminal.



Resultado: a leitura foi realizada corretamente.

4.2 Teste de leitura de arquivo JSON

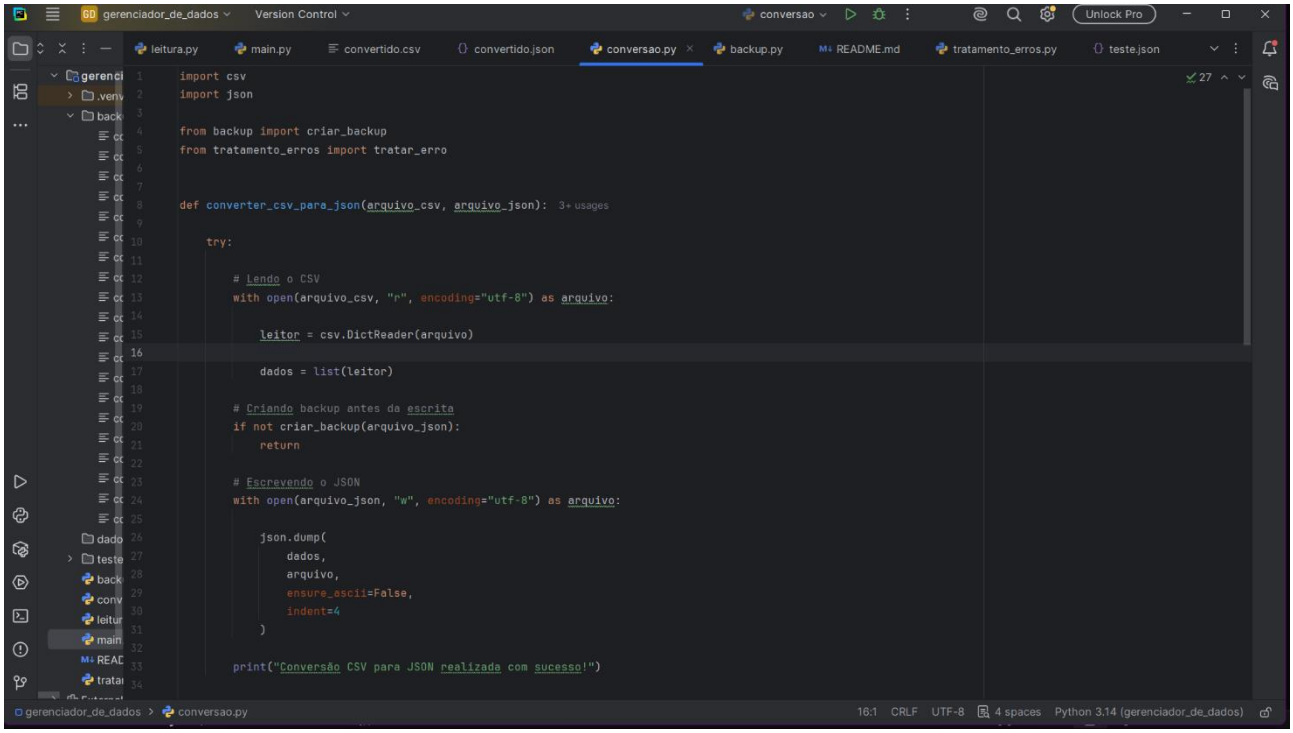
Neste teste, foi verificado se o sistema conseguia acessar e interpretar arquivos no formato JSON. Para isso, foi utilizada a função "ler_json()".



Resultado: os dados do arquivo JSON foram lidos corretamente.

4.3 Teste de conversão de CSV para JSON

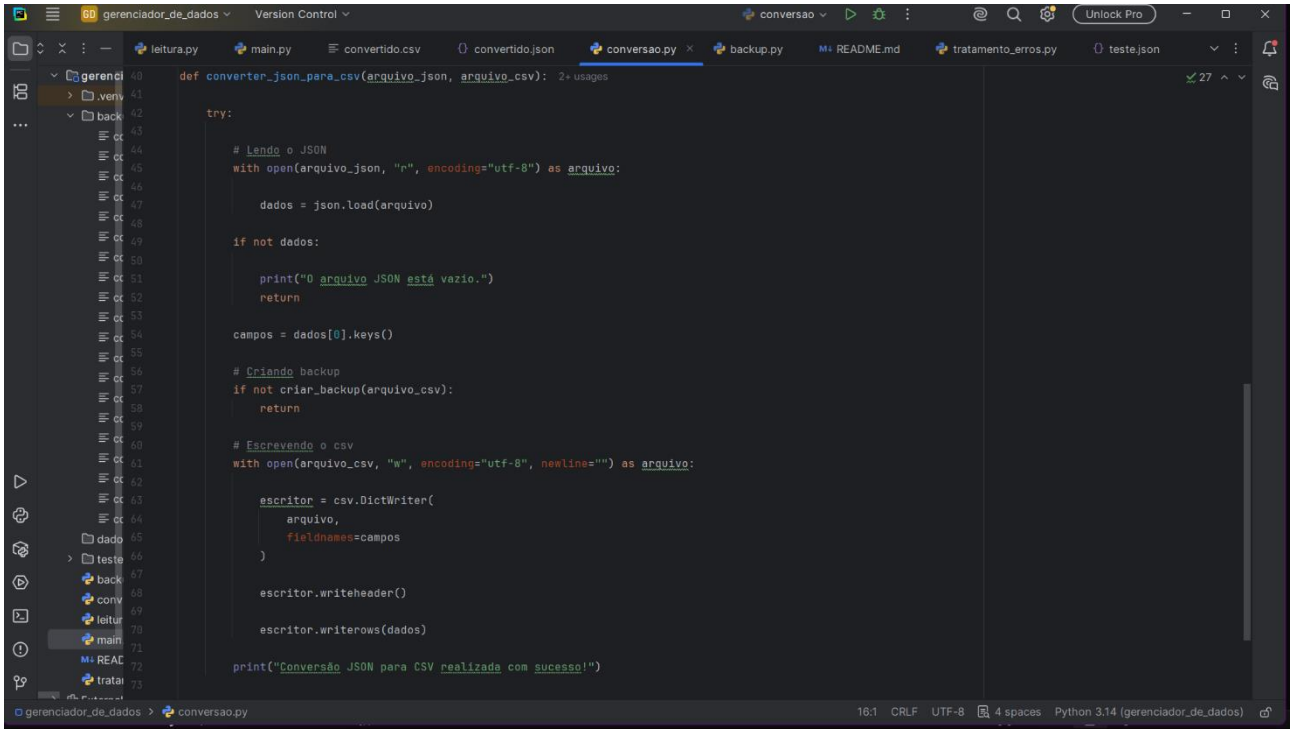
O objetivo deste teste foi verificar a conversão de um arquivo CSV para o formato JSON. A função "converter_csv_para_json()" foi utilizada para realizar a transformação dos dados.



Resultado: o arquivo JSON foi gerado a partir dos dados do arquivo CSV.

4.4 Teste de conversão de JSON para CSV

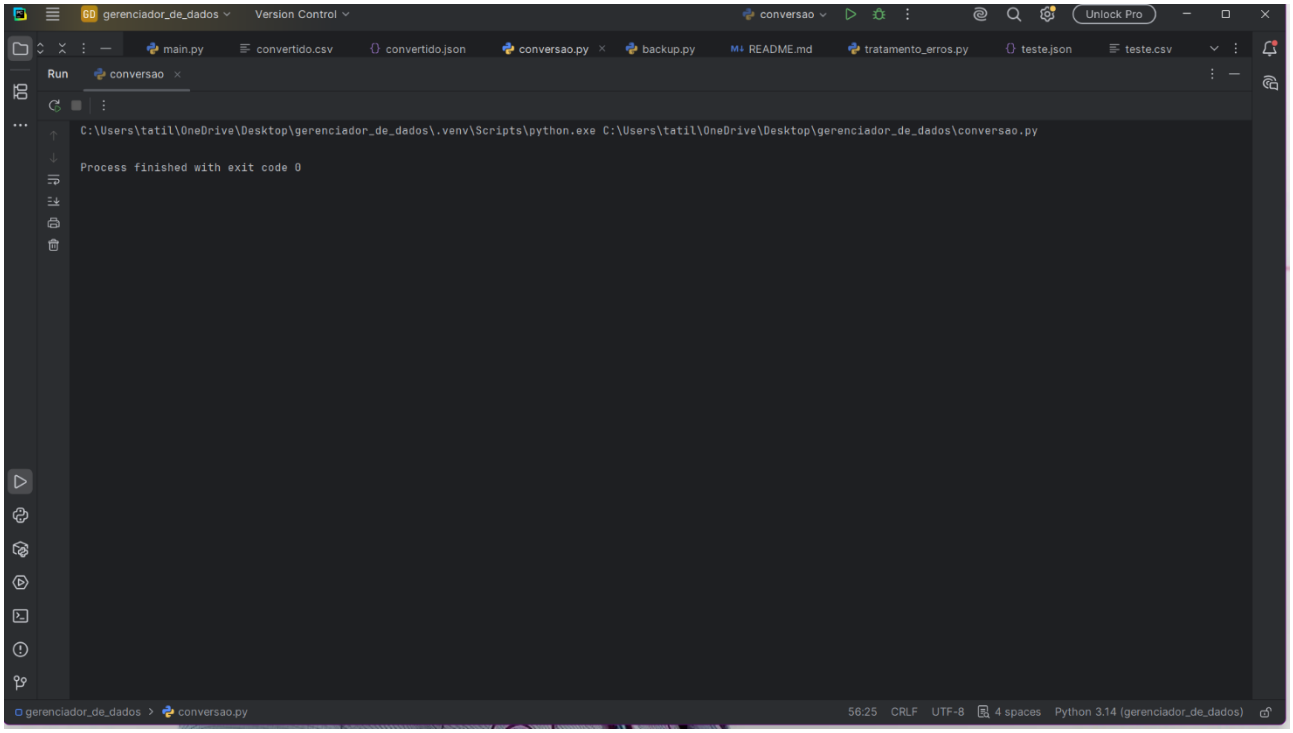
Neste procedimento, foi testada a conversão de um arquivo JSON para o formato CSV, utilizando a função "converter_json_para_csv()".



Resultado: o arquivo CSV foi gerado corretamente a partir dos dados do arquivo JSON.

4.5 Verificação dos arquivos convertidos

Após as conversões, foram realizadas leituras dos arquivos gerados para verificar se os dados estavam presentes e se poderiam ser acessados pelo sistema.

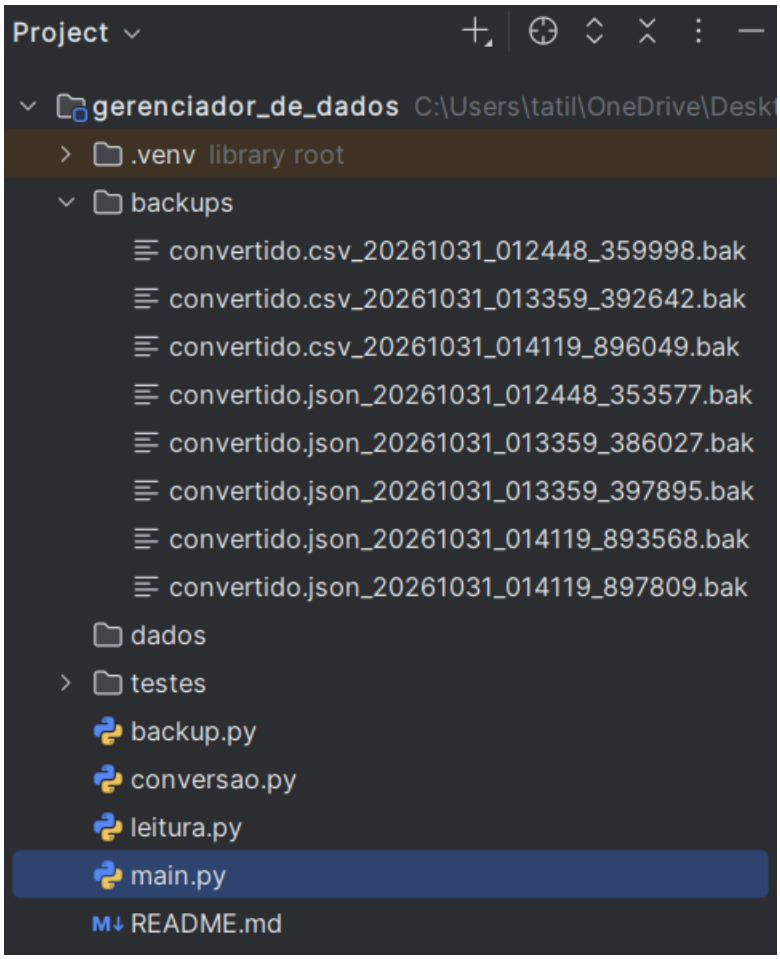


Resultado: os arquivos convertidos puderam ser lidos, permitindo verificar o funcionamento das conversões.

4.6 Teste de criação de backup

Este teste verificou a funcionalidade responsável por criar cópias de segurança dos arquivos antes de uma possível substituição. A função "criar_backup()" foi utilizada para realizar esse procedimento.

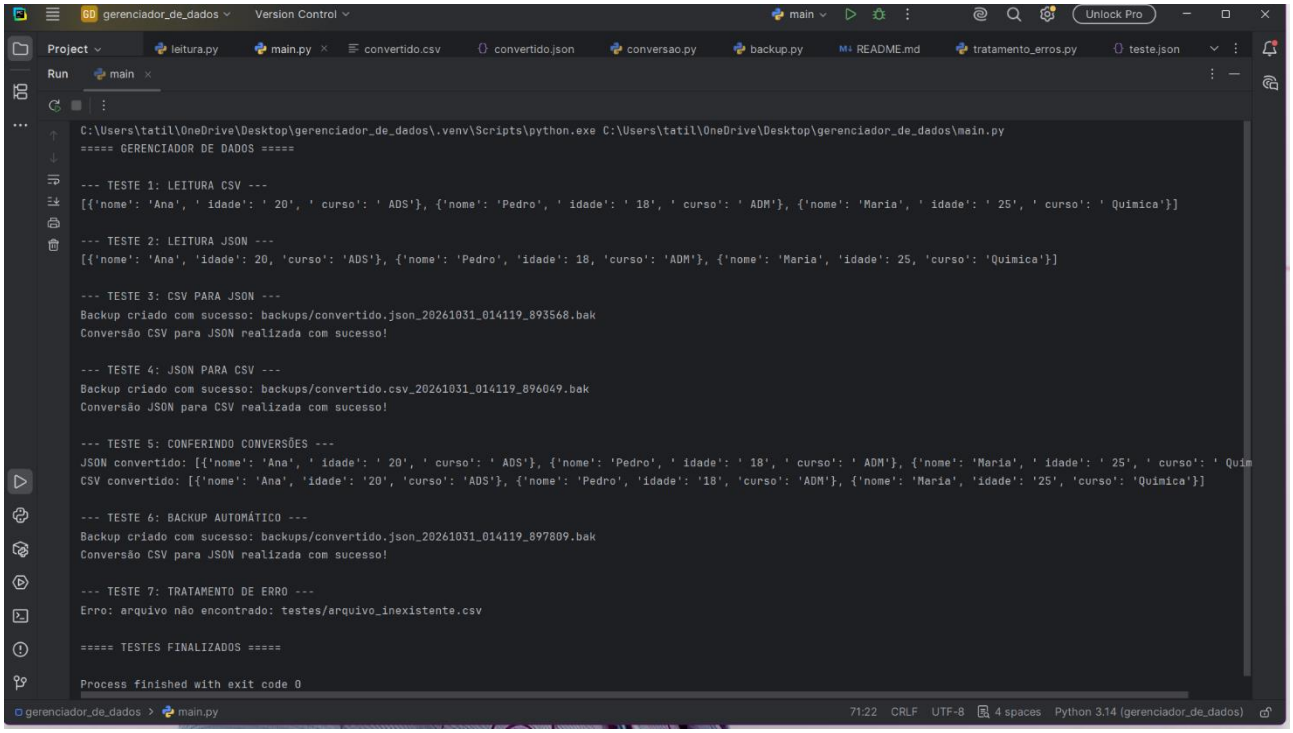
Resultado: o sistema executou a rotina de backup quando identificou um arquivo de destino existente.



4.7 Teste de tratamento de erros

Por último, foi realizado um teste utilizando um arquivo que não existia no diretório indicado. O objetivo foi verificar se o sistema conseguiria identificar a situação e apresentar uma mensagem adequada, sem interromper inesperadamente a execução.

Resultado: o erro de arquivo não encontrado foi tratado corretamente, e o programa conseguiu continuar sua execução.



4.8 Resultado geral dos testes

De modo geral, os testes demonstraram que as principais funcionalidades do Gerenciador de Dados foram executadas conforme o esperado. O programa finalizou sua execução sem apresentar uma exceção não tratada, indicando que a integração entre os módulos ocorreu adequadamente nos cenários testados.

5. CONCLUSÃO

O desenvolvimento do projeto Gerenciador de Dados possibilitou aplicar, na prática, os conhecimentos adquiridos na disciplina de Algoritmos e Programação Avançada, utilizando a linguagem Python para trabalhar com arquivos nos formatos CSV e JSON.

Durante a construção do sistema, foram desenvolvidas funcionalidades de leitura, conversão e manipulação de arquivos, além da implementação de mecanismos de backup e tratamento de erros. A organização do código em módulos também contribuiu para compreender a importância da divisão de responsabilidades e da reutilização de funções em um projeto de programação.

Os testes realizados permitiram verificar o funcionamento das principais funcionalidades implementadas e identificar a importância de tratar possíveis erros durante a execução de um programa.

Conclui-se que os objetivos propostos foram alcançados nos cenários testados. O projeto contribuiu para o aprimoramento dos conhecimentos em Python, especialmente no uso de bibliotecas, manipulação de arquivos, organização de código e desenvolvimento de soluções para problemas práticos.

6. REFERÊNCIAS:

PYTHON SOFTWARE FOUNDATION. Formatos de arquivo. Documentação Python 3. Disponível em: https://docs.python.org/pt-br/3/library/fileformats.html
Acesso em: 30 out. 2026